

Article Title

Jiahuai Mao^{1,2}, Baosheng Li^{2,3}, Third Author^{1,2*}

^{1*}Department, Organization, Street, City, 100190, State, Country.

²Department, Organization, Street, City, 10587, State, Country.

³Department, Organization, Street, City, 610101, State, Country.

*Corresponding author(s). E-mail(s): iiiauthor@gmail.com;
Contributing authors: jhmao.sp@gmail.com; iiauthor@gmail.com;

Abstract

The abstract serves both as a general introduction to the topic and as a brief, non-technical summary of the main results and their implications. Authors are advised to check the author instructions for the journal they are submitting to for word limits and if structural elements like subheadings, citations, or equations are permitted.

1 Introduction

Edge intelligence is changing where machine learning takes place. Increasingly, data are generated by vehicles, robots, unmanned aerial vehicles, mobile sensors, industrial devices and personal edge systems operating in the physical world. These systems observe local environments continuously and often need to improve models close to the data source. Moving all observations to a remote cloud can expose sensitive information, overload wireless links, increase response latency and make learning dependent on infrastructure that may not always be available [1, 2]. Federated learning reduces this dependence by keeping raw data on devices and exchanging model information instead [3, 4]. However, many practical federated systems still rely on a central coordinator that collects updates and redistributes a shared model. In large-scale and mobile edge deployments, this coordinator can become a communication bottleneck, a point of failure and a poor match for intermittent connectivity [5, 6].

Decentralised federated learning removes the permanent coordinator and replaces server-side aggregation with peer-to-peer collaboration [6?]. This change is not only

architectural. It also makes the communication structure part of the learning problem. Without a stable central server, the system must decide who should collaborate with whom, how often information should be exchanged, and how learning should continue when clients join, leave or become temporarily unreachable. These questions are especially important in mobile edge networks, where computation speeds differ across devices and wireless links can be delayed or unstable. At the same time, edge data are naturally heterogeneous. A vehicle, drone, robot or sensor observes only a partial view of the world, shaped by its location, task, hardware and operating condition. As a result, local updates can differ not only in magnitude but also in direction, making unstructured peer-to-peer averaging inefficient or unstable.

This creates a basic tension for decentralised edge learning. On the one hand, models must exchange information often enough to remain aligned. Without sufficient exchange, local training on non-IID data can cause models to drift towards client-specific objectives. Delayed information can further weaken correction because a received model may describe an older training state. On the other hand, frequent communication is costly. It increases bandwidth use, energy consumption and waiting time, and it can make the system fragile when some devices are slow or intermittently connected. The challenge is therefore to maintain useful model alignment in a changing population of heterogeneous devices without imposing continuous global synchronisation.

Existing studies have made important progress on individual aspects of this problem. Decentralised optimisation and gossip learning have shown how communication graphs and mixing rules affect consensus and convergence [6–9]. Asynchronous methods have studied learning when devices progress at different speeds and updates arrive with delay [10?]. Communication-efficient methods reduce exchange cost through sparsification, compression or adaptive communication [11? , 12]. Hierarchical and clustered federated learning exploit locality by aggregating within smaller groups before broader information exchange [? ? ? ?]. In dynamic edge learning, however, grouping is not a neutral preprocessing step. It determines which clients wait for each other, whether cluster models are trained on comparable data, whether leaders can remain connected, and how much communication is needed to realign separated models. These factors are coupled, but they are often treated separately.

Here we study decentralised edge learning from this coupled perspective. We ask how a serverless learning system can organise a changing set of heterogeneous clients into stable collaboration groups, train efficiently within each group, and exchange information across groups without requiring frequent global synchronisation. To address this problem, we introduce XXX, a dynamically clustered decentralised learning framework for heterogeneous mobile edge networks. The framework follows a two-time-scale principle. At a slower topology time scale, active clients are organised into balanced clusters using measurable descriptors of local training time and data composition. At a faster learning time scale, available clients train within each cluster, while cluster leaders periodically exchange model states through a connected overlay.

The design of XXX is guided by the role that clusters play in the learning process. Within a cluster, clients synchronise with the same cluster model, so grouping clients with similar training times reduces waiting. Across clusters, model exchange is more

useful when different clusters are trained on comparable data, so the grouping rule also reduces the mismatch between each cluster’s label distribution and that of the active population. Cluster-size balance is imposed as a hard constraint so that no cluster becomes too small or too large. These choices keep the topology rule simple and measurable while linking it directly to learning stability and system efficiency.

XXX also separates temporary availability changes from persistent topology changes. During a fixed-topology interval, a temporarily unavailable client simply does not participate in the current cluster update. At topology refreshes, persistent membership and link changes are incorporated. The method first attempts to repair the previous topology and reruns full topology construction only when the required cluster-size and connectivity conditions cannot be restored. This reduces unnecessary structural changes while still allowing the system to adapt when the active population or feasible communication graph changes.

After clusters are formed, XXX separates fast within-cluster learning from slower across-cluster model exchange. Within each cluster, available clients train from the current cluster model and upload model changes to the cluster leader, which aggregates them using sample-size weights. Across clusters, leaders periodically mix their models with cached states received from neighbouring leaders. Delayed older messages cannot overwrite newer cached states. This allows information to propagate through the decentralised network without forcing all clusters to synchronise after every local update.

We evaluate XXX across non-IID data partitions, heterogeneous computation profiles, delayed inter-cluster communication and dynamic client membership. The results show that balanced topology construction reduces intra-cluster waiting and cluster-level data mismatch, asynchronous cluster-level learning improves wall-clock convergence, and topology refresh maintains training stability under membership and link changes. Together, these findings suggest a practical design principle for decentralised edge intelligence: stable local collaboration groups should be maintained under changing membership, while broader model exchange should occur at a slower, topology-aware time scale.

2 Results

We evaluated the proposed method under heterogeneous edge-learning conditions, where clients differ in local data distributions, training speeds and availability. The experiments used standard image-classification benchmarks, including MNIST, Fashion-MNIST, CIFAR-10 and CIFAR-100. Unless otherwise stated, client data were partitioned using a Dirichlet distribution with concentration parameter α , where smaller α indicates stronger non-IID data heterogeneity. We report results for $\alpha = 0.1$, $\alpha = 0.3$ and $\alpha = 0.5$, with additional settings provided in Supplementary Results.

The edge system was simulated with N clients. Each client was assigned a local training-time estimate T_i^{loc} , and a feasible communication graph was generated before each clustering round. The training-time estimates were used only for topology construction, as described in Methods. During training, each cluster maintained its own local update counter and progressed independently. Unless otherwise stated,

inter-cluster mixing was performed after every H completed local updates of each cluster.

We compared the proposed method with representative federated and decentralised learning baselines, including synchronous FedAvg, FedProx, decentralised stochastic gradient methods over a fixed peer-to-peer graph, random clustered asynchronous training, clustered training without inter-cluster mixing, and synchronous clustered training. These baselines were selected to separate the effects of balanced topology construction, local synchronisation, asynchronous cluster progress and leader-to-leader model exchange.

We report three groups of metrics. Learning performance was measured by test accuracy, test loss and the wall-clock time required to reach a target accuracy. System efficiency was measured by intra-cluster waiting time, total training time and transmitted bytes. Topology quality was measured by the waiting cost J_T , the distribution cost J_D , the selected number of clusters K , and the connectivity of the leader overlay. All results are reported as the mean over XX independent runs, with shaded regions or error bars indicating one standard deviation.

2.1 Balanced asynchronous clustering accelerates training under heterogeneous edge conditions

We first examined whether balanced asynchronous clustering improves learning under the joint effect of data and system heterogeneity. The main comparison was based on test accuracy as a function of wall-clock time, because wall-clock convergence captures both optimisation progress and system delay.

Figure ??a shows the test accuracy on CIFAR-10 under strong data heterogeneity. The proposed method reached XX% accuracy after XX minutes, whereas FedAvg and FedProx required XX and XX minutes to reach the same accuracy. The advantage was most visible in the early and middle stages of training, where synchronous methods were delayed by slow clients. Decentralised baselines avoided a central server but converged more slowly because model exchange was not organised around balanced cluster-level learners.

The same trend was observed across datasets and heterogeneity levels. Table ?? reports the final test accuracy after a fixed training budget. The proposed method achieved the highest or second-highest accuracy in most settings. Under $\alpha = 0.1$, it improved accuracy by XX percentage points over random clustered asynchronous training and by XX percentage points over decentralised training without balanced topology construction. These gains show that the improvement does not come from asynchronous execution alone, but from combining asynchronous progress with balanced cluster formation.

The benefit was also clear when convergence was measured by the time required to reach a target accuracy. Figure ??b shows that the proposed method consistently reduced time-to-accuracy under both data and system heterogeneity. When the variance of client training times increased, the gap between the proposed method and synchronous baselines became larger. This result confirms that allowing clusters to progress independently reduces the impact of slow clients outside the same cluster.

We further measured communication efficiency by plotting test accuracy against transmitted bytes. As shown in Fig. ??c, the proposed method reached a target accuracy of XX% with XX% fewer transmitted bytes than the decentralised baseline and XX% fewer transmitted bytes than synchronous clustered training. This reduction comes from two design choices: clients communicate mainly within small clusters, and leader-to-leader communication is amortised over the mixing interval H .

Together, these results show that balanced asynchronous clustering improves wall-clock convergence without reducing final accuracy. The improvement is strongest when both data heterogeneity and system heterogeneity are present, which is the target setting of the method.

2.2 The constructed topology reduces both waiting time and distribution mismatch

We next analysed why the constructed topology improves training. A useful topology should satisfy two requirements before model optimisation begins. First, clients in the same cluster should have similar local training times, because each cluster performs synchronous local aggregation. Second, different clusters should have similar overall data compositions, because cluster models are later exchanged through asynchronous leader-level mixing.

Figure ??a compares the class composition of the clusters constructed by the proposed method with the class composition of the active-client population. The proposed construction produced cluster histograms that were close to the active-population histogram across all tested non-IID settings. In contrast, random clustering often produced clusters dominated by a small number of classes, especially when $\alpha = 0.1$. This shows that the distribution term J_D reduces the mismatch between each cluster and the active population.

Figure ??b quantifies this effect using the distribution cost J_D . Under the strongest non-IID setting, the proposed construction reduced J_D by XX% compared with random clustering and by XX% compared with time-only clustering. The gap became smaller as α increased, because the client data were already closer to IID, but the proposed construction remained consistently more balanced.

The proposed construction also reduced the waiting cost caused by within-cluster synchronisation. Figure ??c shows that J_T was lower than that of random clustering and data-only clustering. Thus, the method did not improve data balance by grouping together clients with very different speeds. Instead, it formed clusters that were balanced in both data composition and local training time.

We also examined how the selected number of clusters balanced topology quality and communication overhead. Figure ??d reports J_{bal} , the normalised communication proxy $\bar{J}_C(K)$, and the final topology score J_K for different candidate values of K . Small values of K reduced leader-overlay communication but increased within-cluster waiting. Large values of K reduced cluster size but increased inter-cluster communication. The selected value of K therefore reflected a practical trade-off between cluster balance and communication cost.

These results support the role of topology construction in the method. By reducing both distribution mismatch and synchronisation waiting, the constructed topology

produces cluster-level learners that are more suitable for later asynchronous model exchange.

2.3 Leader-level asynchronous mixing improves cross-cluster knowledge transfer

We then studied the effect of leader-level asynchronous mixing. This experiment tested whether balanced clusters can learn effectively in isolation, or whether periodic cross-cluster model exchange is needed to maintain global learning progress.

Figure ??a compares the full method with a variant that disables inter-cluster mixing. Without inter-cluster mixing, each cluster trained only on its assigned data, and the cluster models gradually diverged. This led to lower final accuracy, especially under strong non-IID data. The degradation was largest when $\alpha = 0.1$, where local data compositions differed most across clients.

Synchronous inter-cluster mixing improved accuracy but increased wall-clock time. As shown in Fig. ??b, faster clusters had to wait for slower clusters before cross-cluster exchange could be completed. The proposed asynchronous mixing avoided this global waiting. Each cluster continued its own local update sequence and used the newest cached neighbour states when a mixing event was triggered.

Figure ??a shows the local update counters of different clusters over wall-clock time. The counters diverged during training, confirming that clusters progressed asynchronously. Slow clusters did not block faster clusters, while periodic leader-level mixing still allowed information to flow across the connected overlay. This behaviour matches the training process described in Methods, where each cluster maintains its own local counter t_k .

The mixing interval H controlled the trade-off between communication and model consistency. As shown in Fig. ??b, small H values improved early accuracy but increased communication. Large H values reduced communication but allowed cluster models to drift further apart. The default value of H provided a stable trade-off across the tested settings.

We further decomposed the total training time into local computation, intra-cluster waiting, communication and idle time. As shown in Fig. ??c, the proposed method substantially reduced idle time compared with fully synchronous FedAvg because different clusters did not wait for each other. Compared with random clustered training, it reduced intra-cluster waiting because clients with similar local training times were more likely to be grouped together. The reduction became larger as the heterogeneity of T_i^{loc} increased.

These results show that leader-level asynchronous mixing is necessary for efficient cross-cluster knowledge transfer. Local cluster training alone is not sufficient under non-IID data, whereas synchronous cross-cluster mixing reintroduces global waiting. The proposed protocol preserves cross-cluster information exchange while allowing clusters to progress independently.

2.4 Topology refresh maintains training stability under dynamic participation

We next evaluated the method under dynamic edge conditions. These experiments tested whether topology refresh can handle persistent membership changes, temporary client unavailability, link failures and delayed inter-cluster communication.

Figure ??a shows the accuracy trajectory when clients joined and left during training. The proposed method maintained stable training after topology changes. Short-term fluctuations appeared immediately after repartitioning, but the model recovered within XX cluster updates. In contrast, keeping a stale topology degraded performance because departed clients and invalid links were not removed from the communication structure.

We compared topology repair with full reconstruction at every refresh. As shown in Fig. ??b, repair achieved similar accuracy to full reconstruction while reducing topology changes and model-transfer overhead. This result indicates that repairing the previous partition is sufficient when the updated active-client set remains close to the previous one. Full reconstruction was needed only when repair could not restore both the cluster-size constraint and the connected leader overlay.

Temporary client unavailability had a weaker effect than persistent membership changes. Figure ??c reports results under different active-client ratios. The proposed method remained stable because temporary unavailability changed only the participating set $A_k^{t_k}$ of the current cluster update and did not immediately trigger repartitioning. Accuracy decreased gradually as the active ratio became smaller, but the method remained more stable than baselines that rebuilt the topology too frequently.

We also tested communication delays between leaders. Figure ??d shows performance under different delay levels. Moderate delays had limited effect because each leader retained the newest received state from each neighbour, and delayed older messages could not overwrite newer cached states. Severe delays slowed convergence, as expected, but the method remained stable under the tested bounded-delay settings.

These results show that separating temporary unavailability from persistent topology changes is important for dynamic edge learning. The method avoids excessive repartitioning, suspends inter-cluster mixing when no connected leader overlay is available, and uses membership-overlap model transfer to reduce disruption after repartitioning.

2.5 The method scales across client populations and system settings

Finally, we evaluated whether the method remains stable across different client populations and parameter settings. This analysis tested whether the observed gains depend on a narrow choice of cluster number, mixing interval or topology score weights.

Figure ??a reports performance as the number of clients increased from XX to XX. The proposed method maintained stable accuracy and reduced wall-clock convergence time across client scales. The advantage over synchronous baselines became more

visible as the client population grew, because larger systems were more likely to contain slow clients that delayed global synchronisation.

We next varied the mixing interval H . Figure ??b shows that very small H values increased communication, whereas very large H values weakened cross-cluster model exchange. The method remained stable across a broad middle range of H , indicating that it does not rely on a single finely tuned mixing interval.

We also varied the topology weights α_T , α_D and λ_C . As shown in Fig. ??c, increasing α_D improved distribution balance but could increase waiting when training speeds were highly heterogeneous. Increasing α_T reduced waiting but could weaken cluster-level data balance under strong non-IID data. The combined score used by the proposed method gave a more stable trade-off than either single-term variant. The communication weight λ_C controlled the preference for smaller or larger leader overlays and mainly affected transmitted bytes rather than final accuracy.

Additional experiments in Supplementary Results evaluate different datasets, non-IID levels, client availability rates and leader-overlay densities. Across these settings, the proposed method consistently improved the trade-off between accuracy, wall-clock time and communication cost. The empirical findings therefore support the main design of the method: balanced topology construction improves the quality of cluster-level learners, asynchronous leader mixing reduces unnecessary waiting, and topology refresh prevents outdated communication structures from being used under persistent system changes.

3 Discussion

4 Methods

4.1 System overview

We consider federated learning over edge clients with heterogeneous local data, training speeds and availability. Fully synchronous training can be delayed by slow clients, whereas fully independent updates make model exchange across the network difficult to coordinate. We therefore use a clustered training structure. Clients synchronise within small clusters, while different clusters progress asynchronously through model exchange between cluster leaders.

The method separates topology construction from model optimisation. At a *clustering round*, the active clients are grouped into balanced clusters, one leader is selected for each cluster, and a communication overlay is formed among the leaders. The resulting topology is reused for multiple *cluster updates*. During this fixed-topology interval, each cluster performs local synchronous updates, while cluster leaders periodically exchange models through the leader overlay without waiting for other clusters. Persistent changes in client membership or feasible links are handled only at topology refreshes.

At clustering round s , let $\mathcal{C}^s$ denote the set of active clients and let

$$\mathcal{V}^s = \{V_1^s, \dots, V_{K^s}^s\}$$

denote their partition into K^s non-overlapping clusters. Client C_i holds a local dataset D_i containing n_i samples and has an empirical objective $F_i(m)$, where m denotes the model parameters. The reference objective associated with cluster k is

$$F_k^s(m) = \sum_{C_i \in V_k^s} \frac{n_i}{n_k^s} F_i(m), \quad n_k^s = \sum_{C_i \in V_k^s} n_i. \quad (1)$$

This objective describes the data assigned to the cluster during the current fixed-topology interval. An individual cluster update may use only the clients that are available at that time.

Each active client reports a compact descriptor containing its dataset size, a summary of its local data composition, and an estimate of the time required for the prescribed local training workload. In the supervised classification experiments, the data descriptor is the normalised class histogram $\mathbf{p}_i$, and the training-time estimate is denoted by T_i^{loc} . These descriptors are used only to choose the communication topology. They do not replace the empirical objectives used during model training.

The complete procedure has three stages. First, the method constructs balanced cluster-level learners by grouping clients according to cluster size, local training time and data composition. Second, it selects one leader from each cluster and keeps only topologies whose leaders form a connected overlay. Third, it trains within each cluster and periodically mixes cluster models across the leader overlay. The topology is refreshed when persistent membership or network changes make the previous topology outdated.

4.2 Balanced topology construction

For topology construction, we use auxiliary costs that are separate from the training objectives in Eq. (1). These costs are used to choose who communicates with whom; model updates are still computed from local empirical objectives.

For a fixed candidate number of clusters K , the number of clients in different clusters is constrained to differ by at most one:

$$|V_k^s| \in \left\{ \left\lfloor \frac{N^s}{K} \right\rfloor, \left\lceil \frac{N^s}{K} \right\rceil \right\}, \quad N^s = |\mathcal{C}^s|. \quad (2)$$

This constraint balances the number of clients assigned to each cluster, but it does not require equal numbers of training samples.

The waiting cost of a partition is

$$J_T = \sum_{k=1}^K \sum_{C_i \in V_k^s} \left(\max_{C_j \in V_k^s} T_j^{\text{loc}} - T_i^{\text{loc}} \right). \quad (3)$$

This term measures the idle time caused by synchronisation inside each cluster. Grouping clients with similar local training times reduces this cost.

Let $\mathbf{p}_G^s$ denote the sample-weighted class histogram of all active clients. The sample-weighted class histogram of cluster k is

$$\mathbf{p}_k^s = \frac{\sum_{C_i \in V_k^s} n_i \mathbf{p}_i}{\sum_{C_i \in V_k^s} n_i}. \quad (4)$$

The distribution cost is then defined as

$$J_D = \frac{1}{K} \sum_{k=1}^K \|\mathbf{p}_k^s - \mathbf{p}_G^s\|_1. \quad (5)$$

This term encourages each cluster to have a data composition close to that of the current active population, so that cluster-level training directions are more comparable.

After normalisation, the two costs are combined as

$$J_{\text{bal}} = \alpha_T \bar{J}_T + \alpha_D \bar{J}_D, \quad \alpha_T + \alpha_D = 1. \quad (6)$$

The normalisation rules are given in Supplementary Method 2.

Finding the globally optimal capacity-constrained partition is combinatorial. We therefore use a deterministic construction that returns one reproducible feasible partition for each candidate K . Clients with the most distinctive class compositions are first placed into different clusters. The remaining clients are inserted according to their increase in J_{bal} , followed by exchange-based refinement. The procedure does not claim to find the global minimum of J_{bal} .

Leader selection is applied after the client partition has been constructed. It is used to test whether the partition can support inter-cluster communication. A topology is feasible only if one leader can be selected from each cluster such that the induced leader overlay is connected. When several connected choices exist, the deterministic procedure prefers the choice with the smallest total local training-time estimate.

Among feasible topologies, the final value of K is selected by combining the balanced topology score with an amortised communication proxy:

$$J_K = (1 - \lambda_C) J_{\text{bal}} + \lambda_C \bar{J}_C(K), \quad 0 \leq \lambda_C \leq 1. \quad (7)$$

Here $\bar{J}_C(K)$ is the normalised communication proxy. The proxy is used only to compare candidate topologies. All communication costs reported in the experiments are measured from the actual transmitted bytes. The deterministic construction, leader selection and communication proxy are specified in Supplementary Method 2.

4.3 Cluster-local learning

Once the topology is fixed, each cluster follows its own sequence of updates. Cluster k maintains a model $m_k^{t_k}$, where t_k is the local update counter of that cluster. Different clusters may therefore have different counters.

At update t_k , let

$$A_k^{t_k} \subseteq V_k^s$$

denote the clients that are available and take part in local training. The cluster leader is also a member of the cluster and performs the same local training when it belongs to $A_k^{t_k}$. In addition, the leader coordinates model distribution, aggregation and inter-cluster communication.

A cluster update can proceed when the current leader is available and $A_k^{t_k}$ is non-empty. The leader sends the current cluster model $m_k^{t_k}$ to the participating clients. Each participating client starts from this model and trains for E local epochs.

Let

$$\Delta_{i,k}^{t_k} = m_{i,k}^{t_k,E} - m_k^{t_k}$$

denote the model change produced by client C_i . The leader aggregates the participating updates as

$$\tilde{m}_k^{t_k+1} = m_k^{t_k} + \eta_{\text{srv}} \sum_{C_i \in A_k^{t_k}} \frac{n_i}{n_{A_k}^{t_k}} \Delta_{i,k}^{t_k}, \quad n_{A_k}^{t_k} = \sum_{C_i \in A_k^{t_k}} n_i. \quad (8)$$

The main experiments use $\eta_{\text{srv}} = 1$.

Equation (8) uses the data of the clients participating in the current update. The cluster objective in Eq. (1) remains the reference objective associated with all clients assigned to the cluster during the fixed-topology interval.

4.4 Asynchronous inter-cluster mixing

A cluster-local update first produces the intermediate model $\tilde{m}_k^{t_k+1}$. If the update does not trigger inter-cluster mixing, this intermediate model becomes the next cluster state. If the update triggers mixing, the leader combines it with the newest cached neighbour states.

Each leader communicates with neighbouring leaders through the connected overlay selected at the clustering round. At the start of a fixed-topology interval, neighbouring leaders exchange their current cluster models to initialise their caches. A symmetric Metropolis matrix $W^s = [w_{kj}^s]$ is then constructed on the leader overlay and retained until the next topology refresh. The matrix W^s specifies the nominal weights used when a cluster performs a mixing event. It does not imply that all clusters synchronously multiply their states by W^s .

After every H completed local updates of cluster k , its leader combines the locally updated model with the newest cached state received from each neighbouring leader:

$$m_k^{t_k+1} = w_{kk}^s \tilde{m}_k^{t_k+1} + \sum_{j \in \mathcal{N}_k^s} w_{kj}^s \hat{m}_{j \rightarrow k}, \quad (9)$$

where $\mathcal{N}_k^s$ is the neighbour set of leader k , and $\hat{m}_{j \rightarrow k}$ is the newest generated state of cluster j received by leader k . At updates that do not trigger inter-cluster mixing,

$$m_k^{t_k+1} = \tilde{m}_k^{t_k+1}. \quad (10)$$

After a mixing event, the new state $m_k^{t_k+1}$ is sent to neighbouring leaders. Each transmitted model carries the sender’s cluster counter. For each neighbour, a leader retains only the received state with the largest sender-side counter. A delayed older message therefore cannot overwrite a newer cached state.

The realised process is asynchronous. Clusters update at different times, only the active cluster changes its model at a given completion event, and cached neighbour states may have been generated earlier. Therefore, the realised influence of each cluster is not determined by the nominal matrix W^s alone. The event-driven representation used for analysis is given in Supplementary Note 1.

4.5 Dynamic membership and topology refresh

Cluster membership and the leader overlay remain fixed within a topology interval. Temporary client unavailability only changes the participating set of the current cluster update and does not immediately trigger repartitioning. A client that does not take part in one update is not removed from its cluster.

At a topology refresh, persistent membership and link changes are incorporated. Clients that have left or remain unavailable are removed from the active-client set, newly joined clients become eligible for clustering, and the feasible communication graph is updated.

The method first attempts to repair the previous partition under the same capacity and connectivity requirements. If repair restores the cluster-size constraint and a connected leader overlay can be formed, the repaired topology is retained. Otherwise, the full topology-construction procedure is rerun over the candidate values of K . The reconstructed topology may use a different number of clusters.

If no connected leader overlay can be formed at a topology refresh, clusters with available leaders continue their local updates, whereas inter-cluster mixing is suspended. A cluster without an available leader pauses its local updates. Topology construction is attempted again at the next refresh. This rule avoids continuing to use a leader overlay that is no longer valid.

When the partition changes, each new cluster is warm-started from the previous cluster models according to membership overlap. A previous cluster contributes more when it contains more retained data from the new cluster. The repair, fallback and model-transfer procedures are given in Supplementary Method 3.

Data availability

Code availability

Acknowledgements

Author contributions

Competing interests

References

- [1] Voigt, P., Bussche, A.: The eu general data protection regulation (gdpr). A Practical Guide, 1st Ed., Cham: Springer International Publishing **10**(3152676), 10–5555 (2017)
- [2] Lim, W.Y.B., Luong, N.C., Hoang, D.T., Jiao, Y., Liang, Y.-C., Yang, Q., Niyato, D., Miao, C.: Federated learning in mobile edge networks: A comprehensive survey. *IEEE Communications Surveys & Tutorials* **22**(3), 2031–2063 (2020)
- [3] McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B.A.: Communication-efficient learning of deep networks from decentralized data. In: *Artificial Intelligence and Statistics*, pp. 1273–1282 (2017). PMLR
- [4] Yang, Q., Liu, Y., Chen, T., Tong, Y.: Federated machine learning: Concept and applications. *ACM Transactions on Intelligent Systems and Technology (TIST)* **10**(2), 1–19 (2019)
- [5] Bonawitz, K., Eichner, H., Grieskamp, W., Huba, D., Ingerman, A., Ivanov, V., Kiddon, C., Konečný, J., Mazzocchi, S., McMahan, B., *et al.*: Towards federated learning at scale: System design. *Proceedings of machine learning and systems* **1**, 374–388 (2019)
- [6] Lian, X., Zhang, C., Zhang, H., Hsieh, C.-J., Zhang, W., Liu, J.: Can decentralized algorithms outperform centralized algorithms? a case study for decentralized parallel stochastic gradient descent. *Advances in neural information processing systems* **30** (2017)
- [7] Koloskova, A., Loizou, N., Boreiri, S., Jaggi, M., Stich, S.: A unified theory of decentralized sgd with changing topology and local updates. In: *International Conference on Machine Learning*, pp. 5381–5393 (2020). PMLR
- [8] Boyd, S., Ghosh, A., Prabhakar, B., Shah, D.: Randomized gossip algorithms. *IEEE transactions on information theory* **52**(6), 2508–2530 (2006)
- [9] Nedić, A., Olshevsky, A., Rabbat, M.G.: Network topology and communication-computation tradeoffs in decentralized optimization. *Proceedings of the IEEE*

106(5), 953–976 (2018)

- [10] Lian, X., Zhang, W., Zhang, C., Liu, J.: Asynchronous decentralized parallel stochastic gradient descent. In: International Conference on Machine Learning, pp. 3043–3052 (2018). PMLR
- [11] Wang, J., Sahu, A.K., Yang, Z., Joshi, G., Kar, S.: Matcha: Speeding up decentralized sgd via matching decomposition sampling. In: 2019 Sixth Indian Control Conference (ICC), pp. 299–300 (2019). IEEE
- [12] Tang, Z., Shi, S., Chu, X.: Communication-efficient decentralized learning with sparsification and adaptive peer selection. In: 2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS), pp. 1207–1208 (2020). IEEE